

MLP for Multi-Class Datasets Analysis: Contrasting Feature and Instance Complexity

Introduction

Multi-class classification is a supervised learning task where the objective is to categorize instances into one of three or more distinct classes. This contrasts with binary classification, which deals with only two classes.

Approaches to Multi-Class Classification

Several strategies can be employed to tackle multi-class classification problems:

- **One-vs-Rest (OvR):** This method involves training a separate binary classifier for each class, distinguishing that class from all others. During prediction, the classifier with the highest confidence score determines the class.
- **One-vs-One (OvO):** In this approach, a binary classifier is trained for every possible pair of classes. The final prediction is made based on a voting mechanism across all classifiers.
- **Extension of Binary Classifiers:** Some algorithms, such as decision trees and neural networks, can be naturally extended to handle multiple classes without requiring transformation strategies.

Models Suitable for Multi-Class Classification

Various machine learning models are well-suited for multi-class classification tasks:

- **Decision Trees:** These models split the data based on feature values, assigning class labels to the leaf nodes.
- **Random Forests:** An ensemble of decision trees that improves classification performance by aggregating the predictions of multiple trees.
- **Support Vector Machines (SVMs):** Originally designed for binary classification, SVMs can be adapted for multi-class tasks using strategies like OvR or OvO.
- **Neural Networks:** Multi-layer perceptrons (MLPs) and other architectures are naturally capable of handling multiple classes due to their flexibility and adaptability.
- **Naïve Bayes:** A probabilistic classifier based on Bayes' theorem, effective for text classification and other applications.
- **K-Nearest Neighbors (KNN):** A non-parametric method that classifies instances based on the majority class among the k -nearest neighbors in the feature space.
- **Gradient Boosting Machines (GBMs):** Ensemble methods that build models sequentially, with each new model correcting the errors of the previous ones.

Choosing the Datasets

The Wine Tasting Dataset

The Wine Tasting Dataset is a small multi-class classification dataset used to predict the quality of wine based on its physicochemical properties. It contains 11 features representing chemical attributes of the wine and 4,898 instances (for red wine). The target variable, quality, ranges from 3 to 8, representing different quality levels.

1. Key Correlations with Wine Quality

- Alcohol has the strongest positive correlation with wine quality (0.44), indicating that higher alcohol content generally results in better quality.

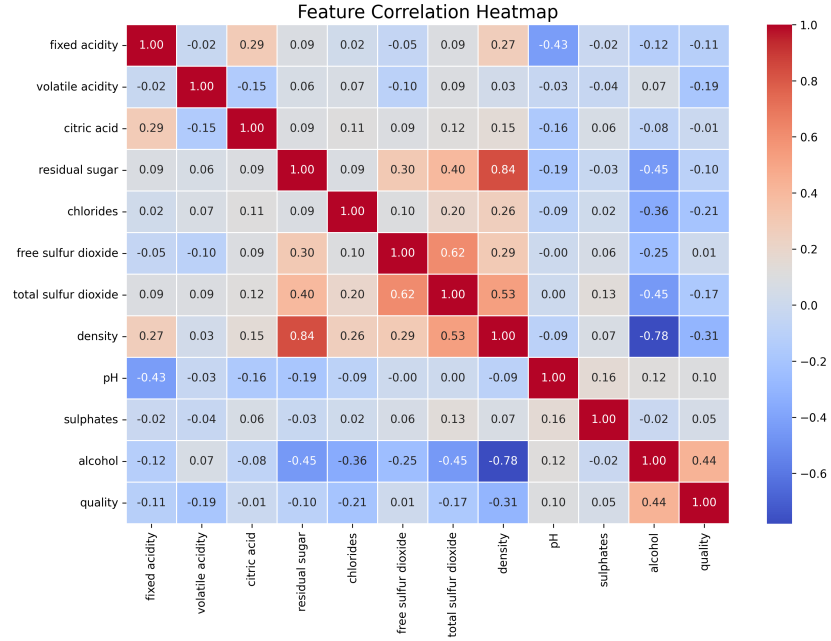


Figure 1: Feature Correlation Heatmap

- Volatile acidity has a negative correlation (-0.19), suggesting that wines with higher acidity are rated lower.

2. Feature Interdependencies

- Residual sugar and density are highly correlated (0.84), meaning denser wines tend to have higher sugar content.
- Free sulfur dioxide and total sulfur dioxide show a strong relationship (0.62), reflecting chemical dependencies.

The bar graph below demonstrates the distribution of wine quality levels in the dataset. It reveals a significant class imbalance, with the majority of instances concentrated around quality levels 5 and 6. These two classes dominate the dataset, while other quality levels, such as 3, 4, 8, and 9, are underrepresented, with quality level 9 having an almost negligible presence.

This class imbalance poses challenges for the model. Since the majority of data belongs to the dominant classes, the model tends to favor these during training, often at the expense of accurately classifying minority classes. This

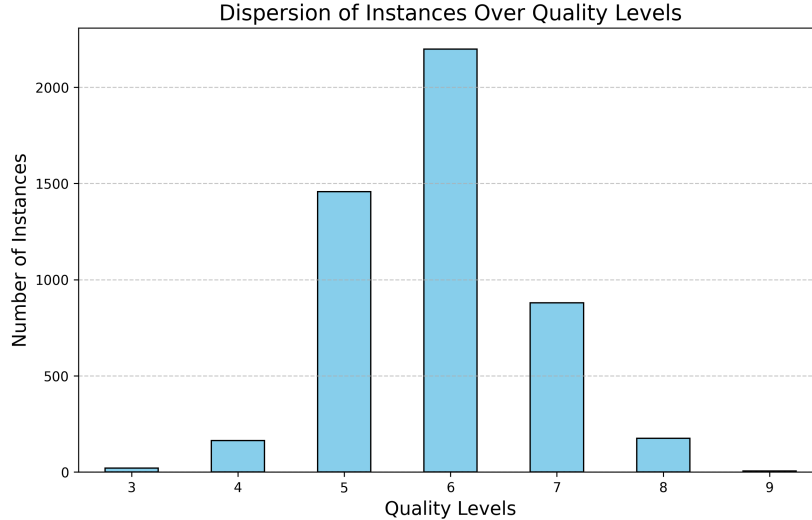


Figure 2: Distribution of Instances Across Wine Quality Levels

bias contributes to the difficulty in achieving high F1-scores, as the model struggles to generalize across all quality levels.

Addressing this imbalance requires preprocessing techniques such as over-sampling minority classes (e.g., using SMOTE) or undersampling majority classes to create a more balanced dataset. Alternatively, assigning class weights in the loss function can help the model penalize misclassifications of minority classes more heavily, potentially improving its performance. This visualization underscores the importance of tackling class imbalance to enhance the model’s ability to classify instances across all quality levels effectively.

The MNIST Dataset

The MNIST Dataset is a benchmark for image classification tasks. It consists of 70,000 grayscale images of handwritten digits (0–9), each represented as a 28x28 pixel grid, totaling 784 features when flattened. The goal is to classify the digit in each image into one of 10 classes.

The bar graph below illustrates the distribution of instances across the digit labels in the MNIST dataset. The dataset is well-balanced, with each digit (0–9) having a roughly equal number of instances, ranging between approximately 5,000 and 7,000 samples per class.

This uniform distribution ensures that the model receives sufficient data

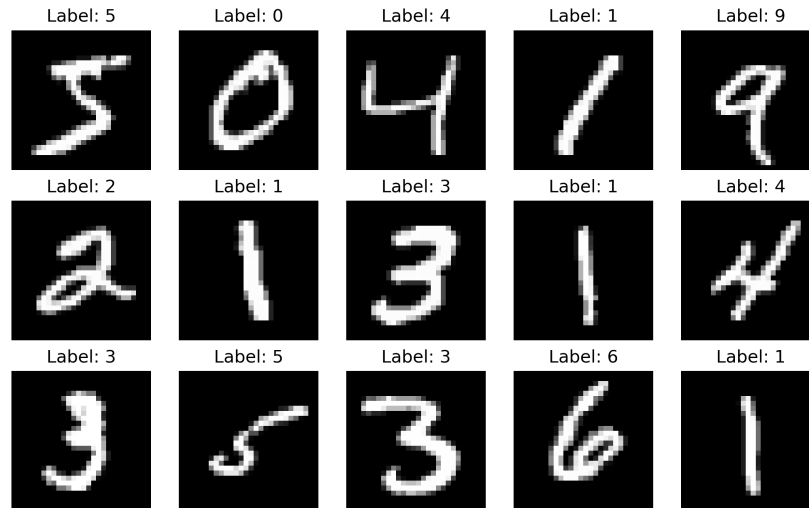


Figure 3: Distribution of Instances Across MNIST Digit Labels

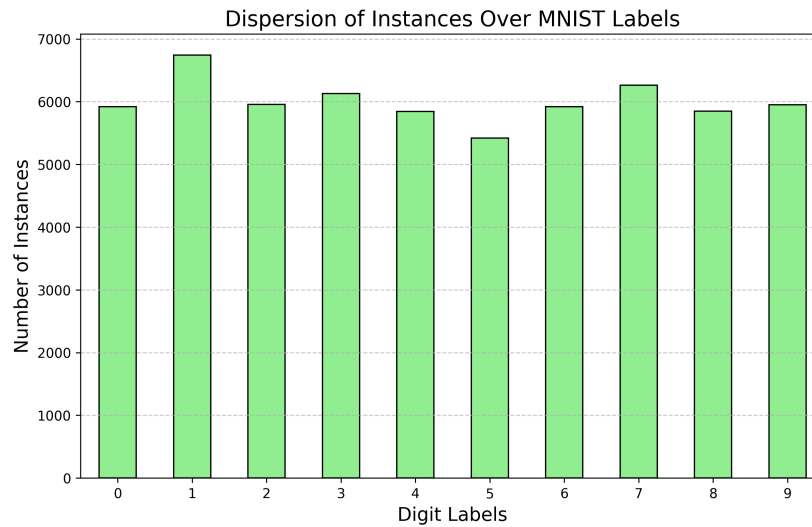


Figure 4: Distribution of Instances Across MNIST Digit Labels

for each class during training, allowing it to learn effectively without bias toward any particular digit. This balance contributes to the high performance of machine learning models on this dataset, as it minimizes issues like class imbalance that can otherwise hinder model generalization.

The Approach

For the Wine Quality Dataset, the optimal approach involves using models like Gradient Boosting or Random Forests. These models are particularly effective for tabular data with relatively few features and instances. Their ability to handle feature importance, capture non-linear interactions, and deal with imbalanced class distributions makes them well-suited for this type of data.

For the MNIST Dataset, the ideal choice would be Convolutional Neural Networks (CNNs). CNNs excel at image classification tasks by capturing spatial relationships and hierarchical patterns directly from raw pixel values. Their convolutional layers are specifically designed to process grid-like data, such as the 28x28 pixel images in the MNIST dataset.

Despite these dataset-specific optimal approaches, we chose to use a Multilayer Perceptron (MLP) as a unified model for both datasets. The MLP is a versatile neural network architecture that can accommodate both tabular and image data. For the Wine Quality Dataset, the MLP processes raw features directly through fully connected layers, while for the MNIST Dataset, the images are flattened into 784-dimensional vectors before being passed into the same fully connected structure.

However, the use of MLP as a common approach comes with challenges. It lacks the specialization of Gradient Boosting for tabular data or CNNs for image data. Consequently, careful tuning of hyperparameters and regularization techniques is required to achieve competitive performance.

Mathematical Principles of the MLP

1. Single Perceptron

A single perceptron is the basic unit of a neural network. It computes a weighted sum of its inputs, adds a bias, and applies an activation function to produce an output.

The mathematical formula for a perceptron is:

$$z = \sum_{i=1}^n w_i x_i + b$$

Where:

- z : Weighted sum of inputs.
- w_i : Weight associated with the i^{th} input.
- x_i : The i^{th} input.
- b : Bias term.

The perceptron's output is computed as:

$$y = f(z)$$

Where:

- $f(z)$: Activation function, such as Sigmoid, ReLU, or Tanh, which introduces non-linearity.

The perceptron updates its weights and bias during training to minimize the error using gradient descent.

2. Multilayer Perceptron (MLP)

An MLP is an extension of a perceptron with multiple layers of neurons, enabling it to learn hierarchical and non-linear representations.

Mathematical Representation of MLP

Input Layer: The input is represented as:

$$X = [x_1, x_2, \dots, x_n]$$

Hidden Layers: For each hidden layer l , the computation involves:

$$Z^{(l)} = W^{(l)} A^{(l-1)} + B^{(l)}$$

$$A^{(l)} = f(Z^{(l)})$$

Where:

- $Z^{(l)}$: Pre-activation values in layer l .
- $W^{(l)}$: Weight matrix for layer l .

- $A^{(l-1)}$: Output (activation) of the previous layer, or input for the first layer.
- $B^{(l)}$: Bias vector for layer l .
- $f(Z^{(l)})$: Activation function applied element-wise.

Output Layer: The final layer computes:

$$Z^{(L)} = W^{(L)}A^{(L-1)} + B^{(L)}$$

$$\hat{Y} = g(Z^{(L)})$$

Where:

- $g(Z^{(L)})$: Activation function for the output layer, such as Softmax for multi-class classification.

End-to-End Formula

The full MLP computation can be expressed as:

$$\hat{Y} = g\left(W^{(L)}f\left(W^{(L-1)}\dots f\left(W^{(1)}X + B^{(1)}\right)\dots + B^{(L-1)}\right) + B^{(L)}\right)$$

This formula represents the nested transformations from the input layer to the output layer.

Building & Testing the Models

The Basic Model

The models for the Wine Quality and MNIST datasets differ slightly in their input and output layers due to the varying number of features and target classes. However, the structure of the hidden layers remains consistent across all models to ensure comparability.

Model Description

- **Hidden Layer Sizes:** A single hidden layer with 128 neurons.
- **Criterion:** CrossEntropyLoss.
- **Optimizer:** Stochastic Gradient Descent (SGD).
- **Learning Rate:** Fixed at 0.01.
- **Epochs:** 30.

Mathematical Principles of SGD and CrossEntropyLoss

Stochastic Gradient Descent (SGD): SGD is an optimization algorithm used to minimize the loss function by iteratively updating the model parameters in the direction of the negative gradient. For a single step, the update rule is given by:

$$\theta = \theta - \eta \cdot \nabla_{\theta} \mathcal{L}$$

Where:

- θ : Model parameters (weights and biases).
- η : Learning rate, controlling the step size.
- $\nabla_{\theta} \mathcal{L}$: Gradient of the loss function with respect to the parameters.

CrossEntropyLoss: This is a loss function used for classification tasks, measuring the dissimilarity between the predicted probability distribution $\hat{y}$ and the true class distribution y . It is defined as:

$$\mathcal{L} = - \sum_{i=1}^C y_i \log(\hat{y}_i)$$

Where:

- C : Number of classes.
- y_i : True probability of class i (1 for the correct class, 0 otherwise).
- $\hat{y}_i$: Predicted probability of class i .

Training Loss Comparison

The graph illustrates the training loss over 30 epochs for both the Wine Quality and MNIST datasets:

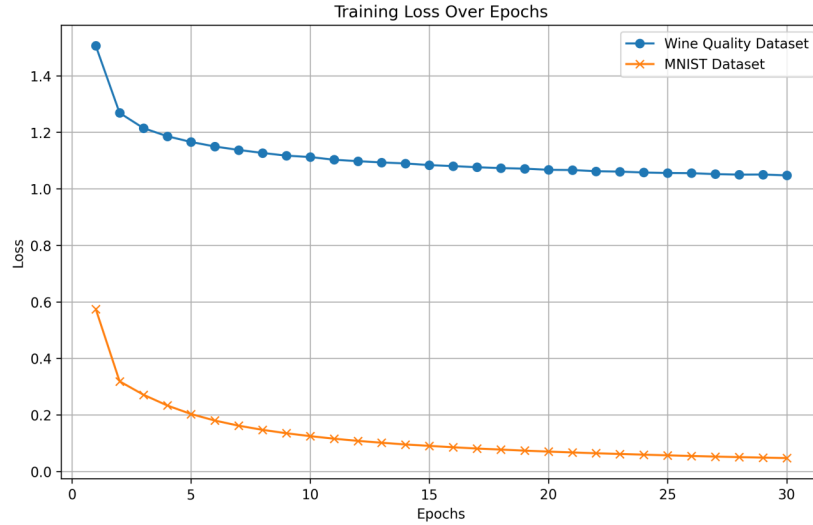


Figure 5: Training Loss Over Epochs

- **MNIST Dataset:** The loss decreases steeply in the first few epochs and flattens as training progresses, demonstrating rapid learning due to the dataset's rich features and balanced class distribution. The final loss approaches zero, indicating strong model performance.
- **Wine Quality Dataset:** The loss decreases more gradually and stabilizes after approximately 20 epochs. The higher final loss reflects the challenges posed by the dataset's class imbalance, limited features, and tabular nature.
- **Comparison:** The sharp contrast between the curves highlights how dataset characteristics, such as feature richness and class balance, significantly impact model training and convergence.

Results

- **Wine Quality Dataset:** Accuracy: 0.5398, F1-Score: 0.5112.

- **MNIST Dataset:** Accuracy: 0.9780, F1-Score: 0.9780.

Reasons for Performance Disparities

Class Imbalance: The Wine Quality dataset is highly imbalanced, with the majority of samples concentrated around quality levels 5 and 6. For example, approximately 65% of the data falls into these two classes. This imbalance skews the model's predictions toward the majority classes, reducing its ability to generalize to minority classes and lowering overall performance.

Lack of Features: The Wine Quality dataset consists of only 11 numerical features, limiting the model's capacity to uncover diverse patterns. In contrast, MNIST provides 784 pixel-based features, offering a much richer input space for learning.

Model Simplicity: The MLP architecture employed uses only one hidden layer with 128 neurons, which may not be sufficient to capture the complex relationships in the Wine Quality dataset. Furthermore, techniques like regularization or dropout, which could improve generalization, have not been implemented.

Why MNIST Performs Better

The MNIST dataset achieves excellent performance, with an accuracy of 97.80% and an F1-score of 97.80%, due to the following factors:

Richness of Input Features: MNIST images provide 784 input features (pixels), offering a wealth of information for the model to learn patterns and distinguish between 10 digit classes.

Balanced Dataset: The MNIST dataset is evenly distributed across the 10 classes, ensuring that the model receives sufficient training examples for each digit, enabling it to generalize effectively.

The Enhanced Model

Moving on from the MNIST model, which achieved optimal results after training, our focus now shifts to improving the performance of the model on

the Wine Quality Dataset. In hopes of increasing the model's accuracy, we employed commonly used optimization techniques in Multilayer Perceptrons (MLPs) and machine learning.

While the MNIST model demonstrated its effectiveness on a well-balanced and feature-rich dataset, the Wine Quality Dataset posed unique challenges due to its limited features and significant class imbalance. This required a tailored approach to address these challenges.

Batch Normalization

Batch Normalization normalizes the inputs to each layer to stabilize the learning process and mitigate internal covariate shift. For a given input x , it computes:

$$\hat{x} = \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$
$$y = \gamma \hat{x} + \beta$$

Where:

- μ : Mean of the mini-batch.
- σ^2 : Variance of the mini-batch.
- ϵ : Small constant for numerical stability.
- γ, β : Learnable parameters for scaling and shifting.

Dropout

Dropout is a regularization technique that randomly sets a fraction of neurons to zero during training. For neuron activations a_i in a layer, it applies:

$$a'_i = \begin{cases} 0 & \text{with probability } p \\ \frac{a_i}{1-p} & \text{otherwise} \end{cases}$$

Where p is the dropout rate.

Adam Optimizer

The Adam optimizer updates parameters using an adaptive learning rate based on moments of gradients. For parameter θ_t , the update rule is:

$$\begin{aligned}m_t &= \beta_1 m_{t-1} + (1 - \beta_1) g_t \\v_t &= \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \hat{m}_t &= \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \\ \theta_t &= \theta_{t-1} - \frac{\eta \hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}\end{aligned}$$

Where:

- g_t : Gradient of the loss at time step t .
- m_t, v_t : First and second moments of the gradient.
- β_1, β_2 : Decay rates for the moments.
- η : Learning rate.
- ϵ : Small constant for numerical stability.

Model Description

- **Hidden Layer Sizes:** Three hidden layers with 256, 128, and 64 neurons, respectively. Each layer is followed by ReLU activation, Batch Normalization, and Dropout with a rate of 0.3.
- **Criterion:** CrossEntropyLoss, suitable for multi-class classification tasks.
- **Optimizer:** Adam.
- **Learning Rate:** Initial learning rate of 0.001 with a StepLR scheduler reducing the rate by a factor of 0.5 every 10 epochs.
- **Epochs:** 30.

Results

Using these techniques, the model achieved:

- **Accuracy:** 0.5857
- **F1-Score:** 0.5727

While this represents a slight improvement, the score remains relatively low, indicating the dataset's limitations and the need for further optimization.

Further Optimization Attempts

After employing various techniques such as designing a deeper and more complex model architecture, feature engineering, the use of class weights in the loss function, and even training an entirely different model using XGBoost, we were unable to achieve a score greater than 0.58 on the Wine Quality Dataset.

However, one final technique, **Synthetic Minority Oversampling Technique (SMOTE)**, proved to be superior. Using SMOTE, we achieved significantly improved scores:

- **Accuracy:** 0.8242
- **F1-Score:** 0.8140

Interpretation of Training Loss Graph

The graph represents the **training loss over 30 epochs** for the Wine Quality Dataset after applying the **SMOTE** technique to address class imbalance.

- **Steady Decline in Loss:** The training loss decreases consistently throughout the training process, indicating that the model is successfully learning and fitting the data. The sharp drop during the first 5 epochs reflects rapid initial learning as the model quickly adjusts to patterns in the data.

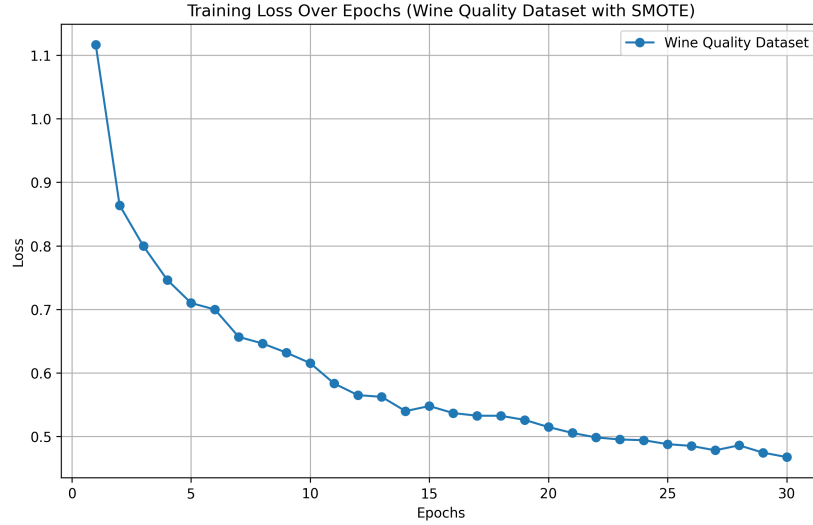


Figure 6: Enter Caption

- **Slower Convergence in Later Epochs:** Beyond epoch 15, the loss begins to plateau, suggesting diminishing returns in learning with additional epochs. This indicates that the model is approaching convergence, and further training may not yield significant improvement.
- **Effectiveness of SMOTE:** The smooth and stable decline in loss demonstrates that the SMOTE technique has provided a more balanced dataset, helping the model generalize better to all classes without overfitting or oscillations during training.

In summary, the graph reflects a well-behaved training process, where the application of SMOTE has enhanced the model's ability to learn effectively from the previously imbalanced dataset.

Principles of SMOTE

SMOTE is a data preprocessing technique designed to address class imbalance in datasets by generating synthetic samples for the minority classes. Unlike random oversampling, SMOTE creates new data points by interpolating between existing instances of the minority class.

How SMOTE Works

Given a dataset with features $\mathbf{X}$ and target labels y , SMOTE works as follows:

1. For each instance x_i in the minority class, identify its k -nearest neighbors (k is a user-defined parameter).
2. Randomly select one of the neighbors, x_j , and generate a new synthetic sample x_{new} using linear interpolation:

$$x_{\text{new}} = x_i + \lambda(x_j - x_i)$$

where $\lambda \in [0, 1]$ is a random number.

3. Repeat this process until the desired number of synthetic samples is generated for the minority class.

Mathematical Representation

If $\mathbf{X}_{\text{minority}}$ represents the feature set of the minority class, the synthetic data points are generated as:

$$\mathbf{X}_{\text{synthetic}} = \mathbf{X}_{\text{minority}} + \lambda(\mathbf{X}_{\text{neighbor}} - \mathbf{X}_{\text{minority}})$$

where:

- $\mathbf{X}_{\text{minority}}$: Original data point from the minority class.
- $\mathbf{X}_{\text{neighbor}}$: Randomly chosen neighbor from the k -nearest neighbors of $\mathbf{X}_{\text{minority}}$.
- $\lambda \sim \mathcal{U}(0, 1)$: Random scalar drawn from a uniform distribution.

Advantages of SMOTE

SMOTE offers several advantages:

- **Balances Class Distribution:** By generating synthetic samples, SMOTE ensures that the dataset has an equal or proportional representation of all classes.
- **Prevents Overfitting:** Unlike random oversampling, which duplicates samples, SMOTE introduces variability by creating new, interpolated samples.

- **Improves Model Generalization:** A balanced dataset allows the model to learn better decision boundaries, improving its performance on minority classes.

Conclusion

In this project, we explored the challenges and solutions for applying Multi-layer Perceptrons (MLPs) to two distinct multi-class classification datasets: the MNIST dataset and the Wine Quality dataset. The MNIST dataset showcased the effectiveness of MLPs on balanced, feature-rich data, achieving optimal results with minimal optimization.

In contrast, the Wine Quality dataset presented unique challenges, including class imbalance and limited feature space. While initial attempts with deeper architectures, feature engineering, and alternative models yielded marginal improvements, the application of **SMOTE** significantly enhanced performance, achieving an accuracy of 0.8242 and an F1-score of 0.8140.

This work highlights the importance of tailored preprocessing techniques like SMOTE and the adaptability of MLPs to various data complexities. It underscores the need to carefully evaluate and address dataset-specific challenges to maximize model performance.